LIDR

Prompts del Módulo 3, para ti

Todo lo que se teclea en el directo, en orden, para que puedas seguirlo o rehacerlo después con calma.

> **Esto no es un guion para pegar a ciegas.** Cada prompt lleva una línea de para qué sirve, y esa línea es lo que hay que entender. Un prompt copiado sin saber qué pide produce código que sale con tu nombre y que no puedes defender.

> ⚠️ **En el directo verás un atajo que aquí NO está, y es a propósito.** Para que la sesión quepa en dos horas, se le pide al agente que no abra PR ni lance el revisor adversarial hasta el final. **Tú no tienes esa prisa.** Deja que tu harness haga su trabajo en cada tarea: eso es lo que vas a hacer en tu equipo, y el atajo del directo es una limitación del formato, no una recomendación.

Antes de nada: haz FORK del repo, no lo clones

https://github.com/LIDR-academy/flowsync-ai4devs

Fork desde la web de GitHub, y después clona **tu copia**:

```
git clone git@github.com:<tu-usuario>/flowsync-ai4devs.git
cd flowsync-ai4devs
git checkout s3/start
make setup
```

⚠️ **El fork no es opcional.** Al final de la sesión le pides a tu harness que cierre la tarea, y eso incluye **abrir un PR**. Sobre un clon directo del repo de la academia no tienes permiso de push, así que ese paso falla. Con tu fork, el PR va a tu propia copia y puedes revisarlo con calma.

La rama `s3/start` viaja en el fork y trae ya el harness del Módulo 1, el login, el PRD y el backlog con sus doce historias. La carpeta `openspec/` **no** está: se crea en el primer paso.

Instalar OpenSpec en el proyecto

Arranca el asistente y elige tu editor. Si usas Copilot o VS Code, leen las mismas skills que instala la opción de Claude Code.

```
openspec init
```

Ver que `specs/` nace vacía

`changes/` es lo que está en curso, `specs/` la verdad sobre cómo se comporta el sistema. Hoy no hay ninguna.

```
ls -R openspec/
```

```
openspec list --specs
```

Escribir la spec de algo que YA existe

Ingeniería inversa sobre el acceso, en las dos capas. **El punto 5 es el que más rinde: no lo quites.**

```
Vamos a documentar como spec de OpenSpec la capability que YA existe en este proyecto: la de cuentas y acceso (registro, inicio de sesión, sesión y perfil). Esto es ingeniería inversa: leer el código y escribir lo que hace HOY, no proponer nada nuevo ni cambiar una línea.
1. Explora el vertical de autenticación COMPLETO, en las dos capas. En `backend/`: sus controladores, el modelo de usuario, sus validadores, sus rutas y sus middlewares. En `frontend/`: las pantallas de acceso, el estado de sesión y la protección de rutas. Ignora todo lo que no sea autenticación.
2. Escribe `openspec/specs/auth/spec.md` con el formato exacto de una spec VIVA de OpenSpec: un `## Purpose` de una o dos frases, y debajo `## Requirements`, con `### Requirement:` en los que el sistema SHALL hacer algo, y bajo cada uno al menos un `#### Scenario:` de cuatro almohadillas con sus dos viñetas `- **WHEN**` y `- **THEN**`. En castellano, salvo las mayúsculas RFC-2119.
3. NO uses secciones ADDED, MODIFIED ni REMOVED: eso es de un delta, y esto no es un delta, es la verdad actual del sistema.
4. Describe solo comportamiento observable desde fuera. Nada de nombres de clase ni de rutas de archivo. Para el backend, observable = la petición y la respuesta HTTP. Para el frontend, observable = lo que una persona ve y puede hacer en pantalla.
5. Al terminar, dime qué comportamiento del código NO has podido expresar como requisito, y por qué.
```

Validar la spec y verla en la CLI

`validate` mira el formato, no el contenido. Que pase no significa que la spec sea correcta.

```
openspec validate auth --strict
```

```
openspec list --specs
```

Versionar

La spec se commitea como cualquier otro artefacto del repo, porque lo es.

Ver qué workflows tienes instalados

Responde con seis. Esa configuración es **global** y no viaja con el repo: quien lo clone no hereda tu perfil.

```
openspec config list
```

Proponer el change de la lista de tareas

Capability nueva, multicapa y que se queda: aquí OpenSpec paga. Nombra las historias por su ruta para que lea los criterios reales.

```
/opsx:propose Dale de alta la capability de tareas del equipo, que todavía no existe. Llama al change exactamente `add-task-list` y a la capability exactamente `tasks` (el delta va en `specs/tasks/spec.md`, no inventes otro nombre). El alcance son cinco historias del backlog, con sus criterios de aceptación: @docs/backlog/E3-actividad-equipo/us-lista-compartida.md y @docs/backlog/E2-gestion-tareas/us-crear-tarea.md, @docs/backlog/E2-gestion-tareas/us-titulo-obligatorio.md, @docs/backlog/E2-gestion-tareas/us-responsable-y-estado-por-defecto.md y @docs/backlog/E2-gestion-tareas/us-cambiar-estado.md. Léelas enteras antes de proponer nada.
Este change es FULL-STACK: la API de AdonisJS en `backend/` Y la interfaz de React 19 en `frontend/`. La interfaz reutiliza los componentes que ya hay en `frontend/src/components/ui/` y el patrón de páginas y rutas que ya existe para el login. No montes design system nuevo ni añadas dependencias.
SIN TESTS: no montes base de pruebas ni escribas tests en este change. Ni tasks.md ni el delta deben incluirlos.
Los tres estados de una tarea viajan por la API como `pending`, `in_progress` y `done`, y se pintan en pantalla como Pendiente, En curso y Hecho. Es un conjunto cerrado: cualquier otro valor se rechaza con 422. NO uses los valores en castellano como identificadores.
Seis restricciones que NO son negociables, aunque no me preguntes por ellas:
1. La tarea NO tiene fecha de vencimiento en este change, y la lista no muestra fechas ni marcas de vencida. No la añadas ni la dejes preparada.
2. La API tiene exactamente tres operaciones: listar todas las tareas, crear una y actualizarla. NO hay lectura individual de una tarea, no hay borrado y no hay endpoints de equipo.
3. Al crear, el título es lo ÚNICO que se pide: el formulario no ofrece ni sugiere responsable, estado ni fecha. La tarea nace en pendiente y con quien la crea como responsable.
4. NO hay regla de orden decidida para la lista. No inventes un criterio de ordenación, no ordenes explícitamente, y anótalo como punto abierto en el proposal.
5. Una sola lista compartida, la misma para todos: no hay tareas privadas ni vista de "mis tareas", y cualquiera puede cambiar el estado y el responsable de cualquier tarea.
6. En la lista, el responsable se identifica por su NOMBRE. Si no tiene nombre puesto se pinta "Sin nombre": nunca su correo ni su id.
Traduce cada criterio de aceptación a scenarios de la spec delta, no lo copies tal cual.
Y antes de escribir, pregúntame lo que de verdad cambie el alcance o el comportamiento observable.
```

Lo que hay que llevar decidido

`propose` no pregunta en abierto: **abre un cuadro de opciones y marca una como recomendada.** No hay nada que pegar, hay que elegir. **Ese cuadro es el momento barato de decidir**, cuando todavía no

existe una línea de código: en una demo se va con las recomendadas por tiempo, en tu equipo no.

Y fíjate en que el prompt de arriba lleva seis restricciones dentro. Ese es el patrón que te llevas: **lo que no puede quedar a merced de una elección en vivo se escribe en el prompt, antes de que el agente genere sus opciones.**

Mirar el change antes de tocar nada

Cuatro archivos y cero líneas de código. El momento más barato de cambiar de opinión.

```
openspec list
```

```
openspec show add-task-list
```

```
openspec validate add-task-list --strict
```

Corregir el contrato antes que el código

Opcional. Va sobre el change, no sobre el código, y esa es toda la lección.

```
/opsx:update add-task-list Antes de implementar, corrige el contrato: [decir aquí el ajuste concreto que se haya visto al revisar]. Toca el delta de specs/ y, si hace falta, tasks.md. No escribas código todavía.
```

Implementar, primera tanda: backend

Se parte en dos: además de poder enseñar algo a mitad, sale más rápido que de una tirada.

```
/opsx:apply add-task-list Haz solo las secciones de backend de tasks.md (modelo de datos y API). Para al terminar la última de backend y no toques el frontend todavía.
Seguimos en la misma rama: no crees una rama nueva, no abras PR y no lances el subagente adversarial-reviewer.
```

Implementar, segunda tanda: frontend

Continúa donde lo dejó, leyendo la checklist del disco. Entre tandas puedes hacer `/compact` sin perder nada.

```
/opsx:apply add-task-list Continúa con las secciones de frontend de tasks.md: la capa de acceso a la API y la interfaz de la lista. La sección de verificación de punta a punta la hago yo a mano en el navegador, no la marques.
Seguimos en la misma rama: no crees una rama nueva, no abras PR y no lances el subagente adversarial-reviewer.
```

Archivar y fusionar en la spec viva

Lo hace la CLI, no el modelo. Después abre la spec y busca `ADDED`: no hay ninguno.

```
openspec archive add-task-list
```

Léelo antes de seguir: dice qué spec actualiza, con qué totales, y **avisa si quedan tareas sin marcar**.

```
openspec list --specs
```

Versionar

Migración, modelo, API, componentes y spec entran en el mismo cambio.

Proponer la fecha de vencimiento

Modifica comportamiento ya escrito. Fíjate en lo corto que es: no hace falta pedirle `ADDED` ni `MODIFIED`, lo deduce leyendo la spec que ya existe.

```
/opsx:propose Añade la fecha de vencimiento y las tareas vencidas, según
@docs/backlog/E2-gestion-tareas/us-fechas-vencimiento.md. Léela entera. Llama al change
exactamente `add-task-due-date`. Backend y frontend, SIN TESTS.
La fecha es de calendario, sin hora: una tarea cuya fecha es HOY todavía no está
vencida, lo estará mañana.
Seis restricciones que NO son negociables, aunque no me preguntes por ellas:
1. Una tarea está vencida si se cumplen las TRES a la vez: tiene fecha, esa fecha es
anterior a hoy, y su estado NO es `done`. Sin fecha no se vence nunca, y una tarea en
`done` nunca está vencida por antigua que sea su fecha.
2. Ese veredicto lo da el BACKEND, y la API lo expone como un campo booleano `isOverdue`
en la representación de una tarea. NO lo resuelva el frontend por su cuenta. Si para
respetar el huso horario de cada persona necesitas su día, que el cliente lo mande y el
servidor devuelva el veredicto; lo que no vale es que la API no diga si una tarea está
vencida.
3. `isOverdue` NO se persiste en ninguna columna: se calcula en cada lectura. Nada de
jobs ni de procesos que marquen tareas por la noche. Y el cliente no puede enviarlo: si
lo envía, se ignora.
4. Se acepta poner una fecha anterior a hoy, al crear y al actualizar: registrar algo
que ya llega tarde es legítimo, nace vencida y no se rechaza. La fecha se puede poner,
cambiar y quitar, y quitarla se hace enviándola explícitamente vacía.
5. La historia dice "al abrir la tarea" y ese endpoint no existe todavía. Este change
añade la lectura individual de una tarea, que es la superficie mínima que la historia
necesita, y lo anota como decisión en el design con su punto abierto. La pantalla de
detalle completa NO entra.
6. La LISTA no cambia lo que muestra: sigue con título, responsable y estado, sin fecha
y sin marca de vencida en pantalla.
Fuera de alcance: notificaciones, recordatorios, recurrencia, ordenar o filtrar por
fecha, y tests de cualquier tipo.
Y antes de escribir, pregúntame lo que de verdad cambie el alcance o el comportamiento
observable.
```

Lo que hay que llevar decidido, segunda ronda

Mismo cuadro de opciones, mismo criterio: se elige, no se pega. Y mismas restricciones dentro del prompt.

Leer el delta

Trae las tres operaciones: `ADDED`, `MODIFIED` y `REMOVED`. La tercera aparece cuando un requisito nuevo **niega** uno anterior, y el delta explica por qué.

```
openspec show add-task-due-date --json --deltas-only
```

El momento humano

Se discute el diseño **antes** de que exista el código. Discutirlo en el PR es discutirlo cuando ya está escrito. Fíjate en que no le dices tú cuál era la alternativa: **se la pides a él**, que es más honesto y suele dar mejor respuesta.

```
Abre el design.md de este change y dime, de las decisiones técnicas que has tomado, cuál es la que más te ha costado y qué alternativa descartaste para quedarte con ella. Defiende tu elección frente a esa alternativa con consecuencias concretas sobre esta spec: qué escenario del delta se rompería con cada una. Si sigues pensando que tu decisión es la buena, no cambies nada.
```

Instalar `verify`, que no viene de serie

El perfil de fábrica (`core`) trae seis workflows y `verify` no está. Es un asistente: elige `Workflows only`, marca `verify`, y acepta cuando te pregunte si lo aplica a este proyecto. **Después reinicia tu editor**, que los slash commands se cargan al arrancar.

⚠️ Esto **te cambia la máquina para todos tus proyectos**, no solo para este. Y ojo con el equipo: el archivo `verify.md` viaja en el repo si lo commiteas, pero el perfil manda — quien corra `openspec update` con perfil `core` **se lo borra**.

```
openspec config profile
```

Implementar la fecha

Mucho más corto que el de la lista: el terreno ya existe.

```
/opsx:apply add-task-due-date
Seguimos en la misma rama: no crees una rama nueva, no abras PR y no lances el subagente adversarial-reviewer.
```

Comprobar el código contra el contrato, ANTES de archivar

Es el único momento en que sirve: el change sigue abierto, así que hay delta contra el que comparar. Devuelve *Completeness*, *Correctness* y *Coherence*, con hallazgos CRITICAL / WARNING / SUGGESTION, y ejecuta typecheck, lint y build de verdad.

⚠️ **Es la IA opinando, no una puerta**: se declara experimental, no bloquea el archivado y está sesgado a callarse. Que pase no garantiza que esté bien.

```
/opsx:verify add-task-due-date
```

Archivar y ver la spec absorber el delta

En la spec viva no queda ni un `ADDED` ni un `MODIFIED`: se lee como si siempre hubiera estado así. El histórico lo guarda el change archivado.

```
openspec archive add-task-due-date
```

```
git diff --stat openspec/specs/tasks/spec.md
```

Cerrar la tarea y dejar que el harness trabaje

Commit, PR y revisor adversarial sobre todo lo hecho. **Tarda unos once minutos y merecen la pena**: en los ensayos encontró dos fallos reales, uno incumpliendo un escenario de la propia spec.

```
Da la tarea por cerrada siguiendo las reglas de proceso de CLAUDE.md: commitea, abre el
PR y pasa el subagente adversarial-reviewer sobre todo el trabajo de esta rama.
```

Si algo no te sale igual

Estos prompts se ejecutaron muchas veces al preparar la sesión, y **el resultado nunca fue idéntico dos veces**: cambia el número de requisitos, cambian las secciones de la checklist, y a veces un defecto que aparece en una ejecución no aparece en la siguiente. Lo que se repite es **la forma**; el número exacto, no.

Y una comprobación que ahorra mucho tiempo: **si la API te responde algo que no cuadra con lo que ves en la base de datos, reinicia el servidor.** Un proceso de desarrollo que lleva horas levantado puede seguir sirviendo el código anterior, y eso hace que algo que funciona parezca roto.